

LAB # 13**CONVOLUTIONAL NEURAL NETWORKS (CNN & CONVNETS)****Objectives:**

- To implement CNN model in Python

Hardware/Software Required:

- Hardware: Desktop/ Notebook Computer
- Software Tool: Python 3.6/3.7
- Software Platform: Google Co-lab

Introduction:

In this lab we will see how we can define a CNN network in python using keras and tensor flow. **Keras** is a powerful open-source python library for developing and evaluating deep learning models. It wraps the efficient numerical computation library **TensorFlow** and allows you to define and train neural network models in just a few lines of code.

Convolutional Neural Network

The convolutional neural networks are inspired from the neural networks, they are formed by neurons that have parameters in the form of weights and biases that can be learned. But the main difference of the CNN is that they make the specific assumption that the entries are images, which allows us to determine certain properties in the architecture to recognize specific aspects in the images. CNNs have many benefits in image recognition and other applications like:

1. Detection using CNN is robust to distortions like change in shape, different lighting conditions, different poses, the presence of partial occlusions, horizontal and vertical shifts, and others.
2. It requires less memory for processing and execution.

CNN Architecture

A CNN architecture is made up of various distinct layers that transform the input volume into an output volume through some functions. The most important layers in CNN are:

1. Convolutional Layer
2. Pooling Layer
3. Fully Connected Layer

Convolutional Layer

The most important layer to the convolutional neural network is the convolutional layer that also gives the network its name. This layer performs an operation called a “**convolution**”. In the context of a CNN, a convolution is a linear operation that involves the multiplication of an input to an array of weights, called a filter or a kernel.

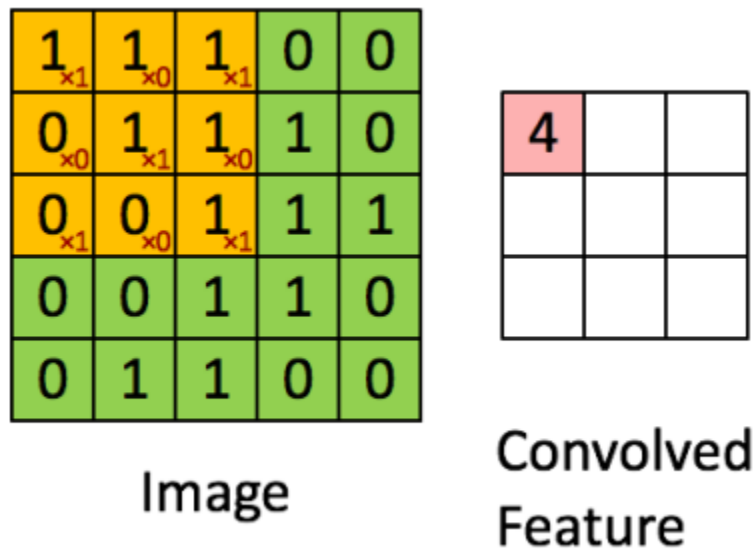


Figure 1: Convolution Operation

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D,Dense,Flatten,MaxPooling2D,Dropout,Activation

model = Sequential()

model.add(Conv2D(32, (3, 3), input_shape=(32, 32, 3), padding='same',
activation='relu'))
```

Building a convolution layer in Keras:

Explanation from the code implementation above:

- The output will have 32 feature maps.
- The kernel size is going to be 3x3.
- The input shape is 32x32 with three channels.
- **Padding** can be valid or same.
 - **Same Padding:** It means the same dimensional output is required as input.
 - **Valid Padding:** It means no padding and it assumes that all the dimensions are valid
- **Activation functions** determine the relevancy of a given node in a neural network. A node relevant to the model's prediction will 'fire' after passing through an activation function. Most common activation function in CNNs is rectified linear function (eg. ReLU). ReLU can be defined as:
 - $\text{ReLU}(x) = \{0, x < 0 \mid 1, x \geq 0\}$
- Which says that nodes with negative values will have their values outputted as 0, denoting them as irrelevant for prediction. These nodes will not activate or 'fire'. Nodes with values equal to or greater than 0 will be left unchanged and will 'fire'.

Another convolutional layer can be added as follows:

```
model.add(Conv2D(64,(3,3),strides=(2,2),activation='relu'))
```

Figure 3: Adding another convolutional Layer

Here we can see the parameter input shape is not mentioned, it only required for the First layer. An additional parameter we can see here is Stride.

Stride: the distance the filter moves at a time. A filter with a stride of 1 will move over the input image, 1 pixel at a time.

Convolutional layer in a convolutional neural network summarizes the presence of features in an input image.

Pooling Layer

Essentially, the pooling layer is used to reduce the dimensionality of the image. It is also used for detecting edges, eyes, nose, corners, and others in the image using multiple filters. There are two ways in which we can achieve pooling:

- Max Pooling: It states the maximum output within a rectangular neighborhood.
- Average Pooling: It states the average output of a rectangular neighborhood.

13	8
79	20

Avg Pooling

Input Image

12	20	30	0
8	12	2	0
34	70	37	4
112	100	25	12
20	30		
112	37		

*Max
Pooling*

Example of Average and Maximum Pooling with stride= 2 and filter size of 2x2

Adding Pooling Layer in Keras

```
model.add(MaxPooling2D(pool_size=(2,2)))
```

Figure 4: Added Max Pooling Layer

- Maximum Pooling is performed on the input with filter size of 2 x 2

Fully Connected Layers

Fully Connected Layer is placed at the end of our CNN architecture to make a prediction, given our learned, convolved features. The Fully Connected layer will take as input a flattened vector of nodes that have been activated in the previous Convolutional layers. The vector input will pass through two to three — sometimes more — dense layers and pass through a final activation function before being sent to the output layer. The activation function used for prediction does not need to be a rectified linear unit. Selecting the right activation function depends on the type of classification problem, and two common functions are:

- **Sigmoid:** is generally used for binary classification problems, as it is a logistic function
- **Softmax:** ensures that the sum of values in the output layer sum to 1 and can be used for both binary and multi-class classification problems.

The final output vector size should be equal to the number of classes you are predicting, just like in a regular neural network.

Implementation of Fully Connected Layer in Keras

```
model.add(Flatten())

model.add(Dense(64,activation='relu'))
model.add(Dense(64,activation='relu'))

model.add(Dense(4,activation='softmax'))

model.summary()
```

Figure 5 : Fully Connected Layers

- 2 fully connected hidden layers with 64 nodes and activation function of relu.
- 1 fully connected output layer with nodes equal to number of classes and activation of softmax.

We have discussed the three important layers in convolutional neural network. But usually when we implement large neural nets on some dataset, they can overfit the training data. Which effects the model learning and results in poor performance when evaluated on testing dataset. To tackle such problem during training, some number of layer outputs are randomly ignored or “**dropped out**”.

Dropout is a regularization method that approximates training many neural networks with different architectures in parallel. Dropout is implemented per-layer in a neural network.

- It can be used with most types of layers, such as dense fully connected layers, convolutional layers, and recurrent layers such as the long short-term memory network layer.
- Dropout may be implemented on any or all hidden layers in the network as well as the visible or input layer. It is not used on the output layer.

Dropout can be added using following command

```
model.add(Dropout(0.20))
```

Figure 6: Adding Dropout Layer in CNN Model

Compiling the Model

Once the

```
model.compile(loss='binary_crossentropy',optimizer='Adam',metrics=['accuracy'])
```

Output layer is reached, the neuron with the highest activation would be the model's predicted class. **Loss** is calculated given the output of the network and results in a magnitude of change that needs to occur in the output layer to minimize the loss.

Neural networks attempt to increase the value of the output node according to the correct class using an optimizer (i.e., gradients). This is done through **backpropagation**.

We decide three key factors during the compilation step:

1. **The optimizer.** Optimizers are algorithms or methods used to change the attributes of your neural network such as weights and learning rate to reduce the losses. The most used optimizer is ADAM in keras. You can also check other optimizers at <https://keras.io/api/optimizers/>.
2. **The loss function:** Loss is nothing but a prediction error of Neural Net. The common loss function for binary class classification is “binary_crossentropy” for binary class classification and for more than two class we can use “categorical_crossentropy” you can also visit <https://keras.io/api/losses/> to check all other loss functions provided in keras
3. **A list of evaluation metrics:** To evaluate the performance of the CNN model. In this lab we will use Accuracy as evaluation metric.

Training A Model

Training a Keras model in literally consists only of calling `fit()` and specifying some parameters. There are a lot of possible parameters, but we'll only supply these:

- The (images `test_acc=model.predict_classes(test_x)` training data and labels), commonly known as X and Y, respectively.
- The number of epochs (iterations over the entire dataset) to train for.
- Batch size refers to the number of training examples utilized in one iteration
- The validation data (or test data), which is used during training to periodically measure the network's performance against data it has not seen before.

Evaluating the Model

Model can be evaluated using following command

```
hist=model.fit(train_x,train_y,batch_size =10,epochs=30,validation_data=(val_x, val_y))
```

And predicted labels can be extracted using following command

Lab Task:**Dataset**

You need to download the dataset from the following link:

https://drive.google.com/drive/folders/1kYLkpupXrjOjani8FymJtXRHgXGbuxwT?usp=share_link

Dataset is already divided into train, test and validate folders.

Steps to Perform

1. Create the simple convnet:
 - create a sequential model
 - add a convolutional layer with 32 channels and 3 x 3 filter. You can use the

following code to do that: `model.add(layers.Conv2D(32, (3,3),
activation='relu', input_shape=(150,150, 3)))`

- The input shape (mentioned above) is required for first layer only
- Add a max polling layer with the following code: model.

```
model.add(MaxPooling2D((2,2)))
```

2. Add three more conv and maxpool layers. Each conv layer should be followed by a maxpool layer. Use 64, 128 and 128 filters of size 3 x 3 for each of these layers followed by a 2 x 2 max pooling layer.
3. Add a flatten layer
4. Finally add a dense layer (Fully Connected layer) with 1024 neurons.
5. Lastly, add a sigmoid layer.

You can view the model summary using `model.summary()`

Model Compilation

1. Use loss : "binary_crossentropy"
2. Metrics: Accuracy
3. Optimizer : Adam

Model Training

1. Train the model with the default learning rate, 30 epochs and batch_size of 15.
2. Plot the training/validation accuracy graph
3. Evaluate the model on test set with the following code: test loss, test acc
`model.evaluate(test images, test labels)`

Now Add a dropout layer with dropout of 0.80 at the end (before fully connected layer) and note its performance on test set again.